

一、(15%) Translate the following C code to MIPS. Assume that the variables i, j, f, g, and h are assigned to registers \$s0, \$s1, \$s2, \$s3, and \$s4, respectively. Assume that the base address of the arrays A and B are in registers \$s6 and \$s7, respectively. Assume that the elements of the arrays A and B are 4-byte words:

$A[i+5] = A[3] - B[j+4];$

```
lw $t0, 12($s6)    # t0 = A[3]
move $t1, $s1       # t1 = j
addi $t1, $t1, 4     # t1 = j+4
sll  $t1, $t1, 2     # t1 = (j+4)*4
add  $t1, $t1, $s7   # t1 = *B[j+4]
lw   $t1, 0($t1)     # t1 = B[j+4]
sub  $t0, $t0, $t1   # t0 = A[3] - B[j+4]
add  $t1, $s0, 5     # t1 = i+5
sll  $t1, $t1, 2     # t1 = (i+5)*4
add  $t1, $t1, $s6   # t1 = *A[i+5]
lw   $t0, 0($t1)     # A[i+5] = A[3] - B[j+4]
```

二、(10%) Translate the following MIPS code to C. Assume that the variables f, g, h, i, and j are assigned to registers \$s0, \$s1, \$s2, \$s3, and \$s4, respectively. Assume that the base address of the arrays A and B are in registers \$s6 and \$s7, respectively.

```
addi $t0, $s6, 20    # t0 = *A[5]
lw   $t0, 0($t0)     # t0 = A[5]
sll  $t0, $t0, 4      # t0 = A[5] * 16
sll  $t1, $s1, 2      # t1 = g * 4
add  $t1, $t1, $s7    # t1 = *B[g]
addi $t1, $t1, 12     # t1 = *B[g+3]
lw   $t1, 0($t1)     # t1 = B[g+3]
sll  $t1, $t1, 2      # t1 = B[g+3] * 4
add  $t0, $t0, $t1    # t0 = A[5] * 16 + B[g+3] * 4
sll  $t2, $s2, 1      # t2 = h * 2
sub  $t0, $t0, $t2    # t0 = A[5] * 16 + B[g+3] * 4 - h * 2
addi $s4, $t0, -4     # j = A[5] * 16 + B[g+3] * 4 - h * 2 - 4
```

ANS: $j = A[5] * 16 + B[g+3] * 4 - h * 2 - 4$

三、(10%) Provide the type and hexadecimal representation of following instruction:

(1)(5%) sw \$s1, 28(\$t0)

(2)(5%) add \$t1, \$s3, \$s5

(1) I-Format: OP (6b) rs (5b) rt (5b) constant (16b)

43 8 17 28

101011 01000 10001 0000 0000 00011100

A D 1 1 0 0 1 C

I-Format

ANS: 0xAD11001C

(2) R-Format: OP (6b) rs (5b) rt (5b) rd (5b) shamt (5b) func (6b)

0 19 21 9 0 32

0000 00 10 011 10101 01001 00000 10 0000

0 2 7 5 4 8 2 0

R-Format

ANS: 0x02754820

四、(15%) Provide the type(5%), assembly language instruction(5%), and binary representation(5%) of instruction described by the following MIPS fields:

op=35, rs=9, rt=22, imm= 4500

(1) I-Format #

(2) lw \$s6, 4500(\$t1) # (lw rt im(rs))

(3) 100011 01001 10110 0001 0001 1001 0100 # (OP rs rt im)

五、(10%) Provide the type(5%) and assembly language instruction(5%) for the following hexadecimal value: 0x8C4A0010_{hex}

0x 8 C 4 A 0 0 1 0

1000 1100 0100 1010 0000 0000 0001 0000

OP = lw rs = 2 rt = 10

(1) I-Format #

(2) lw \$t2, 16(\$v0) #

六、(12%) Assume that we would like to expand the MIPS register file to **256 registers** and expand the instruction set to contain **two times** as many instructions.

How this would this affect the size of each of the bit fields in the **R-format(6%)** and **I-format(6%)** instructions?

reg_bit = $\log_2 256 = 8$

instruction = 1 + 6 = 7 bit

(1) R-Format: \$rs, \$rt, \$rd 暫存器從 5 bit 變 8 bit, OP-Code: 從 6 bit 變 7 bit, Function 從 6 bit 變 7 bit

ANS: (2) I-Format: \$rs, \$rt 暫存器從 5 bit 變 8 bit, OP-Code 從 6 bit 變 7 bit, Immediate 從 16 bit 變 17 bit

七、(10%)For the following C statement, write a minimal sequence of MIPS assembly instructions that does the identical operation. Assume that $\$s0 = a$, $\$s1 = b$, and $\$s2$ is the base address of the array C, and the elements of the arrays C is 4-byte words.

$a = C[b+5] \ll 1;$

```
sll $t0, $s1, 2    # t0 = b * 4
add $t0, $t0, $s2  # t0 = *C[b]
lw  $t0, 20($t0)   # t0 = C[b+5]
sll $s0, $t0, 1    # a = C[b+5] << 1
```

八、(10%)For the following MIPS Code, If we assume we place the loop starting at location 80000 in memory, what is the MIPS machine code for this loop ? (Please fill in table by decimal)

```
80000 Loop: sll $t0, $s2, 5
80004       sub $t0, $t0, $s6
80008       lw  $t1, 0($t0)
80012       bne $t0, $s5, End
80016       addi $s3, $s4, 1
80020       j  Loop
80024 End:
```

	rs	rt	rd	shamt	
0	0	18	8	5	0
0	8	22	8	0	34
35	8	9	0		
5	8	21	20006		
8	20	19	1		
2	20000				

九、(8%) When storing multi-byte data in memory, computers use one of two different byte ordering conventions:

1. Little-endian: The Least Significant Byte (LSB) is stored at the lowest memory address.

2. Big-endian: The Most Significant Byte (MSB) is stored at the lowest memory address.

Example: For the value 0x12345678:

- 0x12 is the MSB (Most Significant Byte, leftmost)
- 0x78 is the LSB (Least Significant Byte, rightmost)

Consider the following MIPS code:

sw \$t0, 0(\$s0)

Assume that register **\$t0** contains the value **0xAABBCCDD** and register **\$s0** contains the address **0x10000000**.

(1) (4%) What value is stored in memory addresses 0x10000000 through 0x10000003 on a **little-endian** machine? Write the byte value stored at each address. Note: express each byte value in **hexadecimal** format.

Address	Value
0x10000000	0xDD
0x10000001	0xCC
0x10000002	0xBB
0x10000003	0xAA

(2) (4%) What value is stored in memory addresses 0x10000000 through 0x10000003 on a **big-endian** machine? Write the byte value stored at each address. Note: express each byte value in **hexadecimal** format.

Address	Value
0x10000000	0xAA
0x10000001	0xBB
0x10000002	0xCC
0x10000003	0xDD